

Exercise 01:

Write a program that reads a Celsius degree and converts it to Fahrenheit.

Formula **$F = (9/5) \times \text{Celsius} + 32$**

```
celsius = float(input("Enter a degree in Celsius: "))
fahrenheit = (9 / 5) * celsius + 32
print(f"{celsius} in Celsius is {fahrenheit} Fahrenheit")
```

Output:

```
Enter a degree in Celsius: 43
43.0 in Celsius is 109.4 Fahrenheit
```

Exercise 02:

Write a program that asks the user to enter the starting velocity v0, the ending velocity v1, and the time span t, then displays the average acceleration with two digits after the decimal point using the formula: **$a = (v1 - v0) / t$**

```
v0 = float(input("Enter v0: "))
v1 = float(input("Enter v1: "))
t = float(input("Enter t: "))
a = (v1 - v0) / t
print(f"The average acceleration is {a:.2f}")
```

Output:

```
Enter v0: 5.5
Enter v1: 50.9
Enter t: 4.5
The average acceleration is 10.09
```

Exercise 03:

Write a program that displays the following on the screen:

```
1 1
2 4
3 9
4 16
5 25
```

You should use the pow() function.

```
for i in range(1, 6):
    print(i, pow(i, 2))
```

Exercise 04:

Write a program that asks the user to enter the radius of a circle and then displays its area and its perimeter.

Area = $\pi \times r^2$

Perimeter = $2 \times \pi \times r$

```
import math
radius = float(input("Enter the radius: "))
area = math.pi * pow(radius, 2)
perimeter = 2 * math.pi * radius
print("Area =", area)
print("Perimeter =", perimeter)
```

Output:

```
Enter the radius: 5
Area = 78.53981633974483
Perimeter = 31.41592653589793
```

Exercise 05:

Write a program that asks the user to enter a character and then displays its ASCII code.

```
ch = input("Enter a character: ")
print("ASCII code =", ord(ch))
```

Output:

```
Enter a character: A
ASCII code = 65
```

Exercise 06:

Given the following:

One birth every 7 seconds

One death every 13 seconds

One new immigrant every 45 seconds

Current population: 1000000

One year: 365 days

Write a program that displays the population for each of the next five years.

```
population = 1000000
births_per_year = (365 * 24 * 60 * 60) // 7
deaths_per_year = (365 * 24 * 60 * 60) // 13
immigrants_per_year = (365 * 24 * 60 * 60) // 45
for year in range(1, 6):
    population = population + births_per_year - deaths_per_year + immigrants_per_year
    print(f"Year {year}: {population}")
```

Output:

```
Year 1: 2861285
Year 2: 4722570
Year 3: 6583855
Year 4: 8445140
Year 5: 10306425
```

Exercise 07:

Write a program that asks the user to enter an integer and then displays whether it is Even or Odd.

```
n = int(input("Enter an integer: "))
match n % 2:
    case 0:
        print("Even")
    case 1 | -1:
        print("Odd")
```

Output:

```
Enter an integer: 12
Even
```

Exercise 08 (Using match):

Write a program that reads an arithmetic operation (+, -, *, or /) and two numbers x and y, then performs the operation $x \text{ op } y$.

```
op = input("Enter an operation: ")
x = float(input("Enter x: "))
y = float(input("Enter y: "))
match op:
    case "+":
        print(f"{x} + {y} = {x + y}")
    case "-":
        print(f"{x} - {y} = {x - y}")
    case "*":
        print(f"{x} * {y} = {x * y}")
    case "/":
        print(f"{x} / {y} = {x / y}")
    case _:
        print("Invalid operation")
```

Output:

```
Enter an operation: *
Enter x: 2
Enter y: 5
2.0 * 5.0 = 10.0
```

Exercise 08 (Using if):

Write a program that reads an arithmetic operation (+, -, *, or /) and two numbers x and y, then performs the operation using if statements.

```
op = input("Enter an operation: ")
x = float(input("Enter x: "))
y = float(input("Enter y: "))
if op == "+":
    print(f"{x} + {y} = {x + y}")
elif op == "-":
    print(f"{x} - {y} = {x - y}")
elif op == "*":
    print(f"{x} * {y} = {x * y}")
elif op == "/":
    print(f"{x} / {y} = {x / y}")
else:
    print("Invalid operation")
```

Output:

```
Enter an operation: -
Enter x: 20
Enter y: 45.5
20.0 - 45.5 = -25.5
```

Exercise 09:

Write a function that computes the sum of the digits in an integer.

For example:

sum_digits(234) returns 9.

Write a test program that prompts the user to enter an integer and displays the sum of its digits.

```
def sum_digits(n):
    total = 0
    while n != 0:
        total += n % 10
        n //= 10
    return total
number = int(input("Enter an integer: "))
print("Sum of digits =", sum_digits(number))
```

Output:

```
Enter an integer: 234
Sum of digits = 9
```

Exercise 10:

A palindromic prime is a number that is both prime and palindromic.

Write a program that displays the first 100 palindromic prime numbers, displaying 10 numbers per line.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
def is_palindrome(n):
    original = n
    reverse = 0
    while n != 0:
        reverse = reverse * 10 + n % 10
        n //= 10
    return original == reverse
count = 0
number = 2
while count < 100:
    if is_prime(number) and is_palindrome(number):
        print(number, end=" ")
        count += 1
        if count % 10 == 0:
            print()
    number += 1
```

Output (First Lines):

```
2 3 5 7 11 101 131 151 181 191
313 353 373 383 727 757 787 797 919 929
```

Exercise 11:

- Create an array of integers named list using the shorthand notation with ten values.
- Write the code to find the sum of all elements in the array.
- Write the code to find the highest value in the array.

```
list = [12, 5, 18, 9, 25, 7, 30, 14, 3, 20]
total = 0
for num in list:
    total += num
print("Sum =", total)
maximum = list[0]
for num in list:
    if num > maximum:
        maximum = num
print("Highest value =", maximum)
```

Output:

```
Sum = 143
Highest value = 30
```

Exercise 12:

Write methods that:

Return the average of an array.

Count the elements above a given key.

Count the elements below a given key.

Find the range (maximum – minimum).

Then write a test program that asks the user to enter ten numbers and displays:

Average

Number of values above average

Number of values below average

Range

```
def average(array):
    return sum(array) / len(array)
def count_above(array, key):
    count = 0
    for num in array:
        if num > key:
            count += 1
    return count
def count_below(array, key):
    count = 0
    for num in array:
        if num < key:
            count += 1
    return count
def find_range(array):
    return max(array) - min(array)
numbers = []
print("Enter 10 numbers:")
for i in range(10):
    numbers.append(float(input()))
avg = average(numbers)
print("Average =", avg)
print("Above Average =", count_above(numbers, avg))
print("Below Average =", count_below(numbers, avg))
print("Range =", find_range(numbers))
```

Output:

```
Enter 10 numbers:
5
7
3
10
8
4
2
6
9
1
Average = 5.5
Above Average = 5
Below Average = 5
Range = 9.0
```

Exercise 13:

Design a class named Triangle that contains:

Three sides (side1, side2, side3)

A default constructor (all sides = 1)

A constructor with three specified sides

area() method

perimeter() method

Then write a test program that creates two objects and displays their area and perimeter.

```
import math
class Triangle:
    def __init__(self, side1=1, side2=1, side3=1):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3
    def perimeter(self):
        return self.side1 + self.side2 + self.side3
    def area(self):
        s = self.perimeter() / 2
        return math.sqrt(
            s * (s - self.side1) * (s - self.side2) * (s - self.side3)
        )
t1 = Triangle()
t2 = Triangle(4, 5, 6)
print("Triangle 1")
print("Area =", t1.area())
print("Perimeter =", t1.perimeter())
print()
print("Triangle 2")
print("Area =", t2.area())
print("Perimeter =", t2.perimeter())
```

Output:

```
Triangle 1
Area = 0.4330127018922193
Perimeter = 3

Triangle 2
Area = 9.921567416492215
Perimeter = 15
```

Exercise 14:

Design a class named Account that contains:

id

balance

annualInterestRate

dateCreated

Include methods for:

Deposit

Withdraw

Monthly interest rate

Monthly interest

Then write a test program that creates an account with:

ID = 1122

Balance = 20000

Annual Interest Rate = 4.5%

Withdraw 2500, deposit 3000, then display the balance, monthly interest, and creation date.

```
from datetime import datetime
class Account:
    def __init__(self, id=0, balance=0):
        self.id = id
        self.balance = balance
        self.annual_interest_rate = 0
        self.date_created = datetime.now()
    def set_annual_interest_rate(self, rate):
        self.annual_interest_rate = rate
    def get_monthly_interest_rate(self):
        return self.annual_interest_rate / 100 / 12
    def get_monthly_interest(self):
        return self.balance * self.get_monthly_interest_rate()
    def withdraw(self, amount):
        self.balance -= amount
    def deposit(self, amount):
        self.balance += amount
account = Account(1122, 20000)
account.set_annual_interest_rate(4.5)
account.withdraw(2500)
account.deposit(3000)
print("Balance: $", account.balance)
print("Monthly Interest: $", account.get_monthly_interest())
print("Date Created:", account.date_created)
```

Output (Example):

```
Balance: $ 20500
Monthly Interest: $ 76.875
Date Created: 2026-07-26 09:00:00
Note: The date and time will vary depending on when you run the program.
```

Exercise 15:

Write your own method to check whether the first string is a substring of the second string.

```
def is_substring(s1, s2):
    return s1 in s2
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")
if is_substring(str1, str2):
    print("The first string is a substring of the second")
else:
    print("The first string is not a substring of the second")
```

Output 1:

```
Enter first string: Java
Enter second string: I love Java programming
The first string is a substring of the second
```

Output 2:

```
Enter first string: Python
Enter second string: I love Java programming
The first string is not a substring of the second
```

Exercise 16 (Inheritance):

Create a class named Person that contains:

A private string data field named name.

A private integer data field named age.

A constructor that initializes name and age.

A method display_info() that displays the person's information.

Create a subclass named Student that extends Person and contains:

A private string data field named major.

A constructor that initializes name, age, and major using super().

Override the display_info() method to display the student's information.

Write a test program that creates a Person object and a Student object and displays their information.

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age
    def display_info(self):
        print("Name:", self.__name)
        print("Age:", self.__age)
class Student(Person):
    def __init__(self, name, age, major):
        super().__init__(name, age)
        self.__major = major
    def display_info(self):
        super().display_info()
        print("Major:", self.__major)
p1 = Person("Ali", 25)
s1 = Student("Ahmad", 21, "Software Engineering")
print("Person Information:")
p1.display_info()
print()
print("Student Information:")
s1.display_info()
```


Output:

```
Person Information:
Name: Ali
Age: 25

Student Information:
Name: Ahmad
Age: 21
Major: Software Engineering
```

Exercise 17 (Polymorphism):

Write a program that demonstrates Polymorphism, Casting, and the instance of concept by creating a list of objects and extracting only the Shape objects.

```
class GeometricObject:
    pass
class Shape(GeometricObject):
    def __init__(self, size, color, filled):
        self.size = size
        self.color = color
        self.filled = filled
    def __str__(self):
        return f"Shape(Size={self.size}, Color={self.color}, Filled={self.filled})"
def get_shapes(objects):
    shapes = []
    for obj in objects:
        if isinstance(obj, Shape):
            shapes.append(obj)
    return shapes
g = []
for i in range(0, 10, 2):
    g.append(GeometricObject())
    g.append(Shape(3 + i, "Red", False))
s = get_shapes(g)
for shape in s:
    print(shape)
```

Output:

```
Shape(Size=3, Color=Red, Filled=False)
Shape(Size=5, Color=Red, Filled=False)
Shape(Size=7, Color=Red, Filled=False)
Shape(Size=9, Color=Red, Filled=False)
Shape(Size=11, Color=Red, Filled=False)
```

Exercise 18 (List):

Write a program that creates a list of integers. The program should:

Add five numbers to the list.

Display all elements.

Remove an element.

Search for a specific number.

```
numbers = [10, 20, 30, 40, 50]
print("List:", numbers)
numbers.pop(2)
print("After removing index 2:", numbers)
n = int(input("Enter a number to search: "))
if n in numbers:
    print("Number exists")
else:
    print("Number does not exist")
```

Output:

```
List: [10, 20, 30, 40, 50]
After removing index 2: [10, 20, 40, 50]
Enter a number to search: 40
Number exists
```

Exercise 19 (Exception Handling):

Write a program that asks the user to enter two numbers and divides them. Handle possible exceptions using try-except.

```
try:
    n1 = int(input("Enter first number: "))
    n2 = int(input("Enter second number: "))
    result = n1 / n2
    print("Result =", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
except ValueError:
    print("Invalid input")
finally:
    print("Program finished")
```

Output 1:

```
Enter first number: 20
Enter second number: 5
Result = 4.0
Program finished
```

Output 2:

```
Enter first number: 20
Enter second number: 0
Error: Cannot divide by zero
Program finished
```

Exercise 20 (Text I/O and File Handling):

Write a program that creates a text file named students.txt, writes student names into the file, then reads and displays the file contents.

```
try:
    with open("students.txt", "w") as writer:
        writer.write("Ali\n")
        writer.write("Ahmad\n")
        writer.write("Sara\n")
    print("File created successfully")
except IOError:
    print("Error writing file")
try:
    with open("students.txt", "r") as reader:
        print("Students:")
        for name in reader:
            print(name.strip())
except FileNotFoundError:
    print("File not found")
```

Output:

```
File created successfully
Students:
Ali
Ahmad
Sara
```

Exercise 21 (Enum) :

Create an enum named Day that contains the days of the week.

Write a program that asks the user to enter a day and displays whether it is a Weekday or Weekend.

```
from enum import Enum
class Day(Enum):
    MONDAY = 1
    TUESDAY = 2
    WEDNESDAY = 3
    THURSDAY = 4
    FRIDAY = 5
    SATURDAY = 6
    SUNDAY = 7
day_input = input("Enter a day: ").upper()
day = Day[day_input]
if day in (Day.SATURDAY, Day.SUNDAY):
    print("Weekend")
else:
    print("Weekday")
```

Output 1:

```
Enter a day: Monday
Weekday
```

Output 2:

```
Enter a day: Sunday
Weekend
```